SUTRA · TRAFFIC INTELLIGENCE

Feature roadmap

Everything in *Features to add in SUTRA* (15 October 2026), turned into an ordered path. Fourteen phases, each one ending in something that runs, is tested, and can be shown to somebody.

15 October 2026 Sarvesh Kumar Yadav Source: featuressutra.pdf
Record: docs/project-record.html

How this is ordered

The source document lists a headline feature and a backlog ranked High / Medium / Low by effort against payoff. This roadmap keeps those priorities but sequences by **dependency**, because two of them cannot be built in the order they were written:

> **A live bug outranks every feature.** Tapping an emergency destination (hospital, fire, police) in Delhi drew a 2,000 km route to Bengaluru. The three chips were hardcoded coordinates with invented names ("Metro Police HQ"). On the emergency path, a wrong destination is worse than a missing one, so this is Phase 0.

> **The surroundings view cannot use Realtime.** The source document's first build step says to subscribe to nearby vehicles. Row-level security lets a driver read only their *own* position row, and Supabase Realtime only delivers rows the subscriber may select — so a driver's subscription returns their own vehicle and nothing else. It needs a server-side function instead, which is why Phase 2 is larger than the document assumes, and why the simulator moves up to Phase 3.

PHASE	WHAT IT DELIVERS	PRIORITY
0	Emergency destinations that are actually nearby	DONE · 15 OCT
1	Shared geometry: bearing, projection, relative position	DONE · 15 OCT
2	Live surroundings view — radar + map toggle	DONE · 15 OCT
3	Vehicle simulator	DONE · 15 OCT
4	Real-device demo run	HIGH
5	Push to a closed app (FCM)	HIGH
6	Trip replay + violations view	HIGH
7	Collision / near-miss warning	MEDIUM
8	Richer violation types	MEDIUM
9	Geofencing + event engine	MEDIUM
10	PostGIS spatial queries	MEDIUM
11	Per-road speed limits	LOW
12	Signal-adapter interface	LOW — PITCH VALUE
13	Offline buffering	LOW

The phases

PHASE 0 **Fix the emergency destinations** BUG DONE · 15 OCT

`PRESETS` in `mobile/app/dashboard.tsx` were static Bengaluru points, and both routing and corridor planning fell back to `here ?? BANGALORE` — so with no GPS fix the app routed from a guessed city centre rather than refusing.

- New Edge Function `supabase/functions/nearby/`: Overpass `nwr(around:R,…)[amenity=hospital|police|fire_station]`, widened through 5 / 15 / 40 km until something is found, sorted by distance. Reuses the mirror list, `Accept: application/json` header and real user agent that the corridor function already needs, because the public Overpass servers answer 406 otherwise.
- Nominatim fallback with `bounded=1` — without it the viewbox is only a soft preference, which is the same bug more quietly.
- Chips become live lookups around the driver, labelled with the real name and distance ("Fortis Hospital · 2.4 km"), disabled until a location is known, cached once per service.
- The `BANGALORE` fallback is deleted from both routing and corridor planning. Without a fix they now refuse.

> **Found on deployment, and it changed the shape of the fix.** The function was deployed and called for a real Delhi coordinate. Overpass answered `406`, its mirror `500`, and Nominatim `403` — while the identical query from an ordinary network returned `200` with the right stations. The public servers refuse the data centre, not the query. The app therefore also retries straight from the phone, exactly as it already does for the corridor's junctions, and the two paths widen the search identically.

Shipped: 15 handler tests, 9 app tests for the lookup and its fallbacks, 5 dashboard tests for the refusal paths. Totals are now 174 database and handler tests, 113 app tests.

PHASE 1 **Shared geometry** DONE · 15 OCT

Small and boring, and it unblocks Phases 2, 7 and 8. `heading` is stored in two tables and **read by nothing**; no bearing function exists anywhere in the project.

- `mobile/src/geo.ts`: `distanceM`, `bearing`, `project(from, bearing, metres)`, `toRelative(ego, other)`. The planar formula shared with the database's `approx_distance_m` lives in one place — the file exists as of Phase 0 and already holds `distanceM`.
- GPS heading is `0` whenever the phone is stationary, so a derived bearing from the previous fix is needed above walking pace.
- Shipped as `mobile/src/geo.ts`: `distanceM`, `bearing`, `project`, `normalizeDeg`, `toRelative` and `headingFrom`, with 16 tests pinning the compass convention and the inverse relationship between bearing and projection.
- The matching SQL `bearing_deg` was *not* added. Every function in the public schema is a REST endpoint, and nothing server-side needs a bearing until Phase 7 — so it is written there instead of shipping an unused one now.

PHASE 2 **Live surroundings view** DONE · 15 OCT

The driver at the centre, other SUTRA vehicles around them, moving smoothly. Radar canvas first, with a toggle to the map-centred version.

Server

A `nearby_vehicles(vehicle_id, radius_m)` security-definer function, polled at the telemetry cadence, because Realtime cannot do this (see above). Each rule gets a test:

- the caller must own the vehicle they are asking as;
- radius clamped, result capped, rows older than ~15 s excluded;
- **no identity leaves the server** — an opaque per-call reference, kinematics and vehicle type only. Never a plate or a driver name. This is what keeps a connected-awareness feature defensible under the DPDP Act, and it is worth saying out loud in a pitch;
- a `warned` flag, so vehicles currently told to give way can be flashed.

Client

- `surroundings.ts` — poll loop and dead reckoning. This is what makes it look live: positions arrive every 1–3 s, so between updates each vehicle is carried forward along its own heading and speed, and a ghost stops moving after about six seconds without news.
- `Radar.tsx` — ego fixed centre, pointing up; range rings at 100 / 200 / 400 m; others placed by *relative* bearing. Built from plain views and Reanimated, which the project already has — no new native module, because a new one cannot be tested before it is installed on a phone.

> **Stated in the view itself,** the way the corridor's SIM badge is: only SUTRA vehicles appear. No pedestrians, no unconnected traffic, no lane detail — that needs sensors on the vehicle. An empty radar means nobody nearby is running SUTRA, not that the road is clear. This paragraph is on the screen, not only in this document.

Shipped: migration `20261015000012_nearby_vehicles.sql` (applied live and rehearsed in a rolled-back transaction against the real project), `surroundings.ts`, `Radar.tsx` and an *Around me* pane with the Radar / Map toggle. Polling follows the telemetry cadence — every second for an authorized emergency vehicle, every three otherwise — and only while the pane is open. 14 database tests, 23 app tests for the module and the component, 5 more for the pane.

PHASE 3 **Vehicle simulator** DONE · 15 OCT

Ranked Medium in the source document, but Phase 2 cannot be *demonstrated* without it: the radar needs other vehicles and there is one phone. Nothing like it exists in the repository today.

A script that signs in, registers vehicles plated `SIM 0001` onward and drives each along a real OSRM route, upserting positions through the same client the app uses. The triggers, violations, corridor warnings and the radar therefore exercise production code — none of them can tell the difference — and row-level security applies exactly as it would to a driver. `--speeder` makes one exceed the limit, which is the fastest way to see an automatic violation alert land on a real phone.

Speed wanders rather than holding constant, and dips toward a crawl now and then: a fleet moving at exactly 38 km/h is the thing that makes a demo look like a demo. Ctrl-C deletes the vehicles; `--cleanup` clears leftovers.

Shipped: `tools/simulator.mjs` (the arithmetic, 16 tests), `tools/simulate.mjs` (the runner). Not yet run against the live project — it signs in as a real account, so that is the owner's to do.

PHASE 4 **Real-device demo run** HIGH

Not code. Install the build, drive a route with the simulator running alongside, record it, and write what was actually observed into the project record. Until this happens, "not verified on a device" stands — and it is the first thing a jury or a traffic authority will ask about.

PHASE 5 **Push to a closed app** HIGH — THE MAIN SAFETY GAP

Every notification today is *local*: an alert arrives over the open websocket and the app posts it itself. A closed app hears nothing, and emergency alerts expire in minutes, so they are already stale by the next launch.

- A device-token table with own-rows-only access, and registration through the existing EAS project id.
- A `notify` function posting to Expo's push service with the existing alert channel and high priority, so the device wakes.
- Fired from the alert trigger through `pg_net` — available on this project but not installed, so the extension needs the same guard the scheduler already uses, or every schema test breaks.

> **Needs you:** FCM V1 credentials — a Firebase project, its service-account key uploaded to EAS, and a new build. It cannot be done from the repository.

PHASE 6

Trip replay and violations

HIGH — CHEAP, DEMOS WELL

- A replay scrubber over a vehicle's stored track in the console, with violation pins and a density view.
- A driver-facing history screen: recent trips, distance, top speed, own violations. Row-level security already permits exactly this.
- **A bug to fix on the way:** the console subscribes to violation inserts, but that table is not in the Realtime publication — so that handler has never once fired.

PHASE 7

Collision / near-miss warning

MEDIUM — STRONGEST SAFETY STORY

A trigger beside the corridor one: for vehicles within a few hundred metres, project both forward and compute time to closest approach; warn both when that is short and close. Dedupe and cool down the way corridor warnings already do — the failure mode here is a phone beeping continuously in slow traffic, so the cooldown and a minimum closing speed *are* the feature, not a detail.

PHASE 8

Richer violation types

MEDIUM

- **Harsh braking** — deceleration against the previous stored position, with the threshold as data rather than code.
- **Wrong direction** — honestly scoped. Without a one-way road layer this is only reliable *against a corridor route*. Doing it for any road needs OpenStreetMap one-way data, so it waits for Phase 11.

PHASE 9

Geofencing and an event engine

MEDIUM

Named zones, and one `events` table with a real taxonomy — zone enter and exit, overspeed, harsh brake, conflict, corridor start and end — which the existing triggers emit into instead of each inventing its own shape. The console's event log becomes a view over one table rather than strings assembled in the browser.

PHASE 10 **PostGIS spatial queries** MEDIUM — AND THE ONE WITH A TRAP

> **The trap:** PostGIS is available on the project but not installed, and the test harness runs Postgres compiled to WebAssembly, which *cannot load it*. The moment a migration says `create extension postgis` unguarded, every schema test breaks.

1. Guard the extension the way the scheduler is already guarded.
2. Add a geography column and spatial index, keep the planar function as the fallback path, and prefer the index when it exists.
3. Verify on a database branch, since it cannot be covered locally — and say so in the README, rather than letting a test count imply coverage that does not exist.

The payoff is real: stored positions grow by thousands of rows per vehicle-hour, and this is the direct answer to "does it scale?".

PHASE 11 **Per-road speed limits** LOW

A speed-zone table seeded from OpenStreetMap `maxspeed`, replacing one global 80 km/h, falling back to the global value where no zone is known. Cleanest once the spatial work of Phase 10 is in place.

PHASE 12 **Signal-adapter interface** LOW EFFORT — HIGHEST PITCH VALUE

The single most useful item for an authority pitch, because it turns "simulated" into "here is where your controller plugs in".

- A written interface: request pre-emption for a junction at a time, confirm, release.
- A function stub with the real shape and a simulation driver that does what the corridor trigger does today.
- A command log recording every instruction that *would* have been sent. A reviewer can then see exactly what SUTRA asks of a city — and that SUTRA never assumed control of anything.

PHASE 13 **Offline buffering** LOW

Queue positions on the phone when the network drops, flush oldest-first on reconnect, capped so it cannot grow without bound. Because the live position is a single upserted row, the queue mainly serves history continuity — a naive flush would replay stale positions as though they were current.

What is deliberately not here

The source document is clear about the limits of the approach, and they belong in the plan as much as the features do:

- **Non-SUTRA vehicles, pedestrians and lanes.** That needs cameras and sensors on the vehicle. SUTRA is cooperative awareness between connected vehicles — a different mechanism, which scales with adoption rather than with hardware.
- **Control of real traffic signals.** Not legally available to a private application in India; it needs a city's adaptive signal control and police authority. Phase 12 documents the join rather than pretending it exists.
- **Live traffic data.** Routing uses speed limits, not congestion. Worth stating whenever "fastest route" is shown.

How each phase finishes

```
cd supabase/tests && npm test     # schema and RLS, Postgres in WASM
cd mobile && npm test && npm run typecheck
cd mobile && npx expo export      # catches missing peers before an EAS queue
```

Then: apply the migration and rehearse the live path inside a rolled-back transaction, deploy any changed function, redeploy the console, and rebuild the app only at the end of a group of phases that changed it.

Standing rule: every phase also updates `docs/project-record.html` and the README's feature, data-model and test-count tables. The record is the document an outside reader is given.
